

## Session 5 : Lumière et matériaux

### 💡 Objectifs de la session

Comprendre la mathématique derrière l'éclairage — la loi de Lambert et le produit scalaire  $\vec{N} \cdot \vec{L}$  — puis explorer les sources de lumières de Babylon.js et le modèle spéculaire de Phong/Blinn-Phong.

### 💡 Rappel : pourquoi la lumière est essentielle

Rappelez-vous la Session 3 : le Fragment Shader calcule la couleur de chaque pixel. Mais sans lumière, un objet 3D n'est qu'un bloc uniforme — tous les pixels ont la même couleur, peu importe la géométrie. C'est la lumière qui révèle la forme, la profondeur et la texture des objets. L'éclairage est le pont entre la géométrie (les normales vues en Session 1) et l'apparence finale (la couleur calculée par le shader).

### 💡 Le rappel critique : les normales

En Session 1, nous avons défini les **normales** — des vecteurs perpendiculaires à chaque face, pointant vers l'extérieur. Ces normales sont l'information essentielle que le GPU utilise pour calculer l'éclairage. Sans normales correctes, la lumière n'a aucun sens. Le produit scalaire ( $\vec{N} \cdot \vec{L}$ ) entre la normale de surface et la direction de la lumière détermine l'**intensité** de la lumière reçue par cette surface.

## Partie 1 : L'Éclairage Mathématique — La Loi de Lambert

### 1. Le problème : combien de lumière reçoit une surface ?

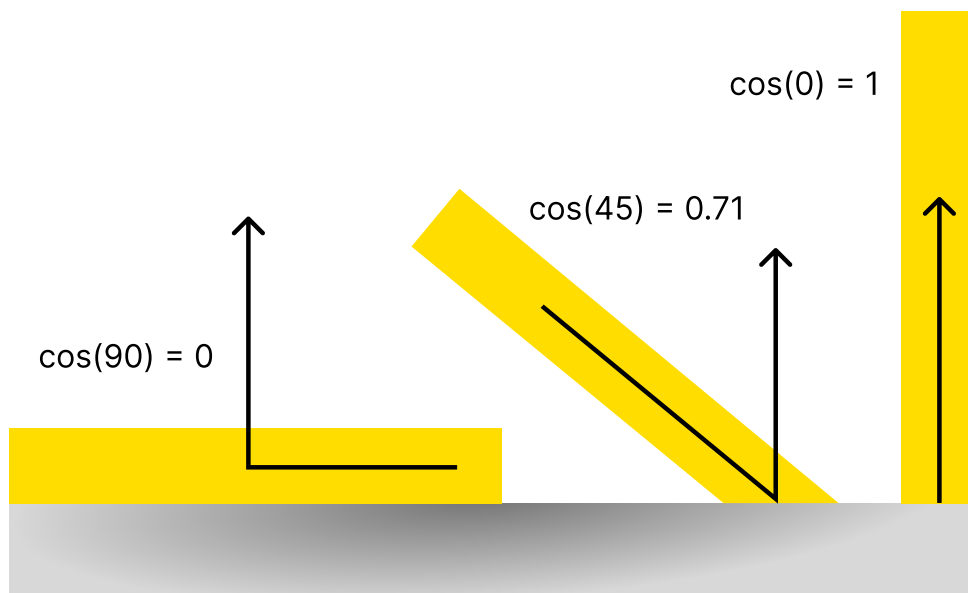


Figure 1: Loi de Lambert : L'intensité est proportionnelle au cosinus de l'angle entre la normale  $\vec{N}$  et la direction de la lumière  $\vec{L}$ .

## L'intuition physique

Imaginez une feuille de papier blanche tenue face au soleil. Si vous l'inclinez progressivement, elle devient de plus en plus sombre. Pourquoi ? Parce que la même quantité de lumière se répartit sur une **plus grande surface** projetée — chaque  $\text{cm}^2$  reçoit moins d'énergie. C'est la **loi de Lambert** : l'intensité reçue est proportionnelle au cosinus de l'angle entre la normale de la surface et la direction de la lumière (voir Figure 1).

## La loi de Lambert (1760)

Pour une surface mate (diffuse), l'intensité lumineuse reçue vaut :

$$k_d = \max(\vec{N} \cdot \vec{L}, 0)$$

où  $\vec{N}$  est la **normale unitaire** de la surface et  $\vec{L}$  la **direction unitaire de la surface vers la lumière**. Le  $\max(\dots, 0)$  coupe les valeurs négatives : une surface dos à la lumière ne reçoit rien.

## 2. Le produit scalaire, au cœur de tout

### Rappel : le produit scalaire

Le produit scalaire de deux vecteurs  $u$  et  $v$  s'écrit :

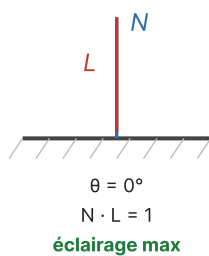
$$u \cdot v = \|u\| \|v\| \cos(\theta)$$

Si  $u$  et  $v$  sont **normalisés** ( $\|u\| = \|v\| = 1$ ), alors :

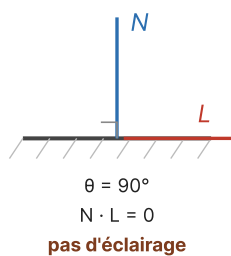
$$u \cdot v = \cos(\theta)$$

C'est la formule fondamentale de toute l'informatique graphique temps réel : un simple produit scalaire donne le cosinus de l'angle entre deux vecteurs.

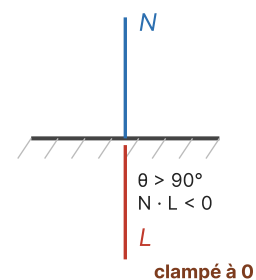
(a) Surface face à la lumière



(b) Lumière rasante



(c) Surface dos à la lumière



Les trois cas de la loi de Lambert :  $N$  (bleu) est la normale,  $L$  (rouge) est la direction de la lumière.

Figure 2: Les trois cas de la loi de Lambert : (a) surface face à la lumière, (b) lumière rasante, (c) surface dos à la lumière.

### Les trois cas de la loi de Lambert

- Si  $\vec{N}$  et  $\vec{L}$  pointent dans la même direction (surface face à la lumière) :  $\theta = 0$ ,  $\vec{N} \cdot \vec{L} = 1$ , éclairage **maximal**.

- Si  $\vec{N}$  est perpendiculaire à  $\vec{L}$  (lumière rasante) :  $\theta = 90^\circ$ ,  $\vec{N} \cdot \vec{L} = 0$ , **pas d'éclairage**.
- Si  $\vec{N}$  pointe à l'opposé de  $\vec{L}$  (surface dos à la lumière) :  $\theta > 90^\circ$ ,  $\vec{N} \cdot \vec{L} < 0$ , mais on utilise  $\max(\dots, 0) \rightarrow$  **zéro** (voir Figure 2).

C'est pourquoi une sphère éclairée présente un dégradé continu du côté éclairé vers le côté sombre : chaque fragment a une normale différente, donc un  $\vec{N} \cdot \vec{L}$  différent.

### 3. La couleur finale : multiplication lumière $\times$ matériau

#### La formule complète du diffus

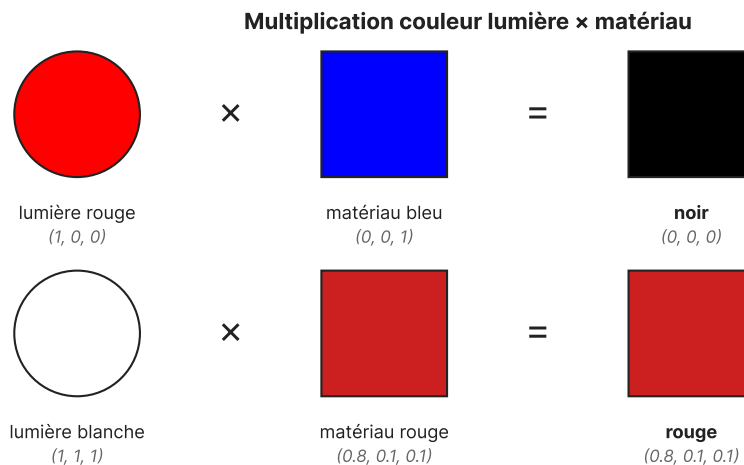
La couleur finale d'un fragment éclairé par la lumière diffuse est :

$$\text{color}_{\text{frag}} = \text{color}_{\text{light}} \times \text{color}_{\text{material}} \times \max(\vec{N} \cdot \vec{L}, 0)$$

- $\text{color}_{\text{light}}$  : couleur de la source (ex. blanc (1, 1, 1)).
- $\text{color}_{\text{material}}$  : couleur intrinsèque de la surface (ex. rouge (1, 0, 0)).
- $\max(\vec{N} \cdot \vec{L}, 0)$  : le facteur d'angle de Lambert.

#### 💡 Astuce : la couleur de la lumière se multiplie

La couleur de la lumière (diffuse) se **multiplie** avec la couleur du matériau. Une lumière rouge (1, 0, 0) sur un matériau bleu (0, 0, 1) donne... du noir (0, 0, 0) ! Le bleu ne réfléchit pas le rouge. C'est la physique réelle : un objet bleu sous lumière rouge apparaît noir (voir Figure 3).



*Un objet bleu sous lumière rouge apparaît noir : le bleu ne réfléchit pas le rouge.*

Figure 3: La couleur finale est le produit composante par composante de la couleur de la lumière et de la couleur du matériau.

### 4. Écriture en GLSL : le shader de Lambert

*Fragment shader minimal (Lambert seul).*

```
// Varyings interpolés depuis le vertex shader
in vec3 vNormal;           // Normale du fragment (espace monde)
in vec3 vLightDir;         // Direction de la lumière vers le fragment

uniform vec3 uLightColor;   // Couleur de la lumière
```

```

uniform vec3 uMaterialColor; // Couleur du matériau

out vec4 fragColor;

void main() {
    // 1. Normaliser (les varyings ne sont plus unitaires après interpolation)
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);

    // 2. Produit scalaire = cos(theta), clampé à [0, 1]
    float diffuse = max(dot(N, L), 0.0);

    // 3. Couleur finale = lumière × matériau × facteur d'angle
    vec3 color = uLightColor * uMaterialColor * diffuse;

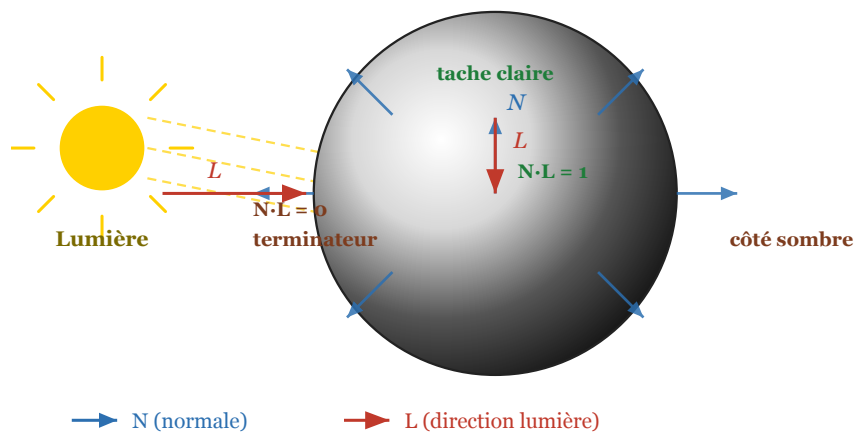
    fragColor = vec4(color, 1.0);
}

```

### ❗ Pourquoi normaliser dans le fragment shader ?

Les normales sont interpolées **linéairement** entre les sommets du triangle. Cette interpolation ne préserve pas la norme : un vecteur unitaire aux sommets ne l'est plus forcément au milieu du fragment. Il faut donc **toujours** appeler `normalize(N)` dans le fragment shader, même si les normales étaient unitaires dans le vertex shader. Oublier cette étape est un bug classique qui donne un éclairage trop terne ou trop brillant.

## 5. Pourquoi Lambert fait un dégradé sur une sphère



*Le dégradé sur la sphère vient de la variation de  $N \cdot L$  : chaque fragment a une normale différente.*

Figure 4: Une sphère éclairée : chaque fragment a une normale radiale différente. Au centre éclairé  $\vec{N} \cdot \vec{L} = 1$  (tache claire), au terminator  $\vec{N} \cdot \vec{L} = 0$ , et le côté opposé reste sombre.

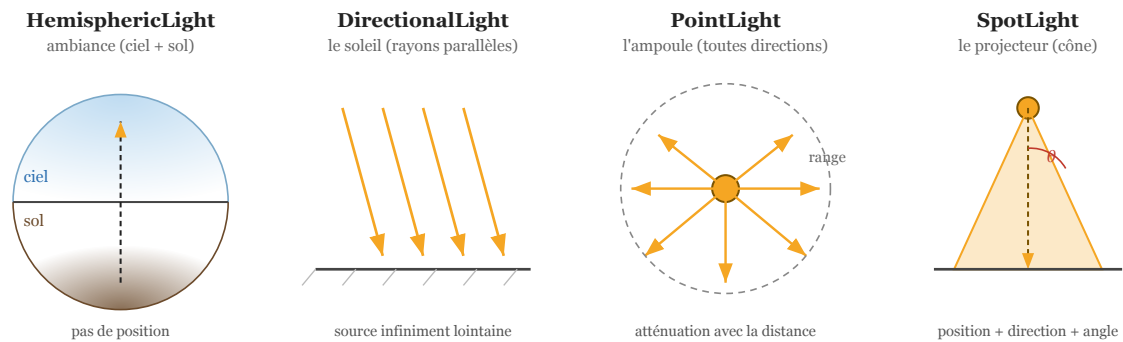
### Géométrie d'une sphère éclairée

Sur une sphère, chaque fragment a une normale différente : elle pointe radialement vers l'extérieur. Au centre de la face éclairée,  $\vec{N}$  et  $\vec{L}$  sont alignés ( $\vec{N} \cdot \vec{L} = 1$ ) → tache la plus claire.

Au bord,  $\vec{N}$  est perpendiculaire à  $\vec{L}$  ( $\vec{N} \cdot \vec{L} = 0$ )  $\rightarrow$  ombre. Entre les deux,  $\vec{N} \cdot \vec{L}$  varie doucement de 1 à 0, créant le dégradé caractéristique (voir Figure 4). C'est exactement ce que reproduit le shader ci-dessus, fragment par fragment.

## Partie 2 : Sources de Lumières et Spéculaire

### 6. Les quatre types de lumières dans Babylon.js



*Les quatre types de lumières intégrées de Babylon.js.*

Figure 5: Les quatre types de lumières intégrées de Babylon.js : HemisphericLight (ambiance), DirectionalLight (soleil), PointLight (ampoule), SpotLight (projecteur).

#### HemisphericLight — La lumière ambiante

Simule la lumière du ciel et du sol. Elle n'a pas de position — elle éclaire toutes les surfaces en fonction de leur orientation (la normale).

- **Ciel** : Couleur pour les surfaces pointant vers le haut (groundColor inversé).
- **Sol** : Couleur pour les surfaces pointant vers le bas.
- **Direction** : Définit quel axe sépare le "haut" du "bas".
- **Usage** : Lumière d'ambiance de base, évite que les zones non éclairées soient totalement noires.

*Créer une HemisphericLight.*

```
const hemiLight = new BABYLON.HemisphericLight("hemi", new BABYLON.Vector3(0, 1, 0), scene);

// Couleur du ciel (surfaces vers le haut)
hemiLight.diffuse = new BABYLON.Color3(0.8, 0.9, 1.0); // Bleu pâle

// Couleur du sol (surfaces vers le bas)
hemiLightgroundColor = new BABYLON.Color3(0.3, 0.2, 0.1); // Brun

// Couleur spéculaire
hemiLight.specular = new BABYLON.Color3(0, 0, 0); // Pas de reflet spéculaire

hemiLight.intensity = 0.6;
```

## DirectionalLight — Le soleil

Lumière parallèle qui simule une source infiniment lointaine (comme le soleil). Tous les rayons arrivent dans la même direction. Elle n'a pas de position — seule la direction compte.

- **Direction** : Un vecteur normalisé qui indique la direction des rayons.
- **Pas de portée** : Éclaire toute la scène uniformément.
- **Usage** : Soleil, lumière naturelle, ombres nettes.

*Créer une DirectionalLight.*

```
const sunLight = new BABYLON.DirectionalLight("sun", new BABYLON.Vector3(-1, -2, -1), scene);

sunLight.diffuse = new BABYLON.Color3(1, 0.95, 0.8); // Lumière chaude
sunLight.specular = new BABYLON.Color3(1, 1, 1);      // Reflet blanc
sunLight.intensity = 1.2;

// La direction est normalisée automatiquement par Babylon.js
// mais il est recommandé de la normaliser soi-même :
sunLight.direction.normalize();
```

## PointLight — L'ampoule

Lumière ponctuelle qui émet dans toutes les directions à partir d'une position dans l'espace. Plus un objet est loin, moins il est éclairé (atténuation).

- **Position** : Emplacement de la source dans la scène.
- **Range** : Portée maximale de la lumière. Au-delà, l'intensité tombe à zéro.
- **Atténuation** : L'intensité diminue avec la distance selon une courbe.
- **Usage** : Ampoule, torche, feu, lampe de poche (sans cône).

*Créer une PointLight.*

```
const bulb = new BABYLON.PointLight("bulb", new BABYLON.Vector3(0, 3, 0), scene);

bulb.diffuse = new BABYLON.Color3(1, 0.9, 0.7); // Lumière chaude
bulb.specular = new BABYLON.Color3(1, 1, 1);
bulb.intensity = 1.0;
bulb.range = 15; // Portée de 15 unités

// On peut animer la position pour simuler un mouvement
scene.registerBeforeRender(() => {
    bulb.position.x = Math.sin(performance.now() * 0.001) * 3;
});
```

## SpotLight — Le projecteur

Lumière conique qui combine une position et une direction, avec un angle d'ouverture. C'est la lumière la plus complexe des quatre.

- **Position** : Origine du cône.
- **Direction** : Axe central du cône.
- **Angle** : Angle d'ouverture du cône (en radians, demi-angle).

- **Exponent (penumbra)** : Contrôle le flou du bord du cône. Plus la valeur est élevée, plus le bord est net.
- **Range** : Portée maximale.
- **Usage** : Lampe torche, projecteur de scène, phare de voiture.

Créer une *SpotLight*.

```
const spotlight = new BABYLON.SpotLight(
    "spot",
    new BABYLON.Vector3(0, 5, 0), // Position (en haut)
    new BABYLON.Vector3(0, -1, 0), // Direction (vers le bas)
    Math.PI / 4, // Angle : 45 degrés
    2, // Exponent (penumbra)
    scene
);

spotlight.diffuse = new BABYLON.Color3(1, 1, 0.8);
spotlight.intensity = 2.0;
spotlight.range = 20;
```

## 7. Paramètres communs à toutes les lumières

### Propriétés partagées

- `light.diffuse` : Couleur principale de la lumière (`Color3`). C'est la couleur qui sera mélangée avec la couleur diffuse du matériau.
- `light.specular` : Couleur du reflet spéculaire (`Color3`). Souvent blanc, mais peut être coloré pour des effets créatifs.
- `light.intensity` : Multiplicateur d'intensité (float). Valeur par défaut : 1.0.
- `light.range` : Portée de la lumière (float). Utilisé par `PointLight` et `SpotLight`. Ignoré par `HemisphericLight` et `DirectionalLight`.
- `light.direction` : Direction de la lumière (`Vector3`). Utilisé par `HemisphericLight`, `DirectionalLight`, et `SpotLight`.

## 8. La lumière spéculaire (Modèle de Phong)

### Au-delà du diffus : la brillance

La loi de Lambert explique les surfaces **mates** (bois, papier, béton). Mais une bille de verre, une voiture peinte ou un métal poli réfléchissent aussi la lumière comme un miroir — c'est la composante **spéculaire**. Elle ne dépend pas seulement de l'angle  $\vec{N} \cdot \vec{L}$ , mais aussi de la position de la **caméra** : le reflet se déplace quand on tourne autour de l'objet.

### Le reflet : $\vec{R} \cdot \vec{V}$

La composante spéculaire simule le reflet brillant d'une lumière sur une surface lisse. Elle dépend de trois vecteurs :

- $\vec{L}$  : Direction de la surface vers la lumière.
- $\vec{R}$  : Rayon réfléchi (le miroir de  $\vec{L}$  par rapport à la normale  $\vec{N}$ ).
- $\vec{V}$  : Direction de la surface vers la caméra.

$$k_s = \max(\vec{R} \cdot \vec{V}, 0)^{\text{shininess}}$$

L'exposant "shininess" contrôle la taille du reflet :

- Valeur faible (ex: 2) : reflet large et flou (plastique mat).
- Valeur élevée (ex: 128) : reflet petit et net (métal poli).

### Le vecteur réfléchi $\vec{R}$

Le rayon réfléchi se calcule par la formule :

$$\vec{R} = 2(\vec{N} \cdot \vec{L})\vec{N} - \vec{L}$$

Géométriquement : on décompose  $\vec{L}$  en une composante normale  $(\vec{N} \cdot \vec{L})\vec{N}$  et une composante tangente. La réflexion inverse la composante tangente, donc on double la composante normale et on soustrait  $\vec{L}$ . C'est la loi de la réflexion de Descartes — l'angle d'incidence égal l'angle de réflexion (voir Figure 6).

*Note* : cette formule n'a de sens que lorsque  $\vec{N} \cdot \vec{L} > 0$  (surface face à la lumière). Quand  $\vec{N} \cdot \vec{L} < 0$ , la surface est dos à la lumière et ne reçoit rien — le shader masque alors la spéculaire avec  $\text{step}(0.0, \text{NdotL})$ .

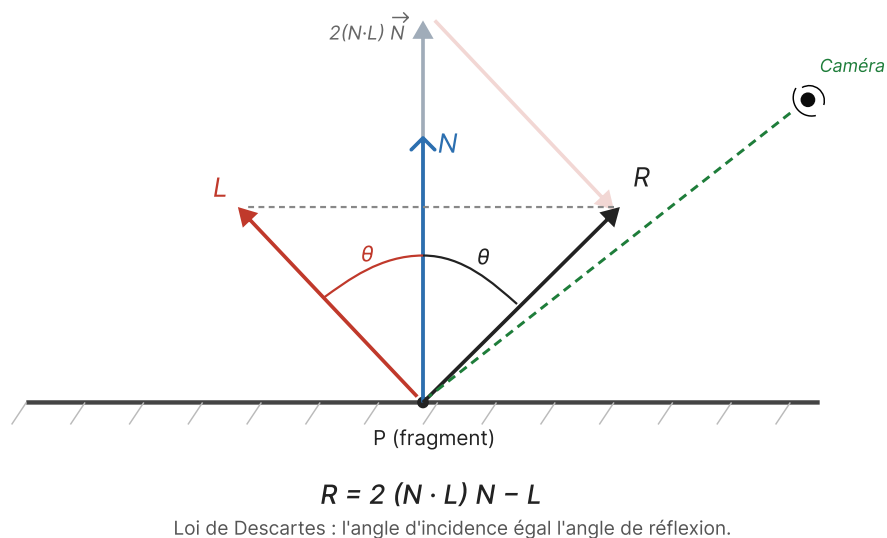


Figure 6: Le vecteur réfléchi  $\vec{R} = 2(\vec{N} \cdot \vec{L})\vec{N} - \vec{L}$  : symétrique de  $\vec{L}$  par rapport à la normale  $\vec{N}$ . L'angle d'incidence  $\theta$  égal l'angle de réflexion.

## 9. Blinn-Phong : la variante optimisée

### Le vecteur half-angle $\vec{H}$

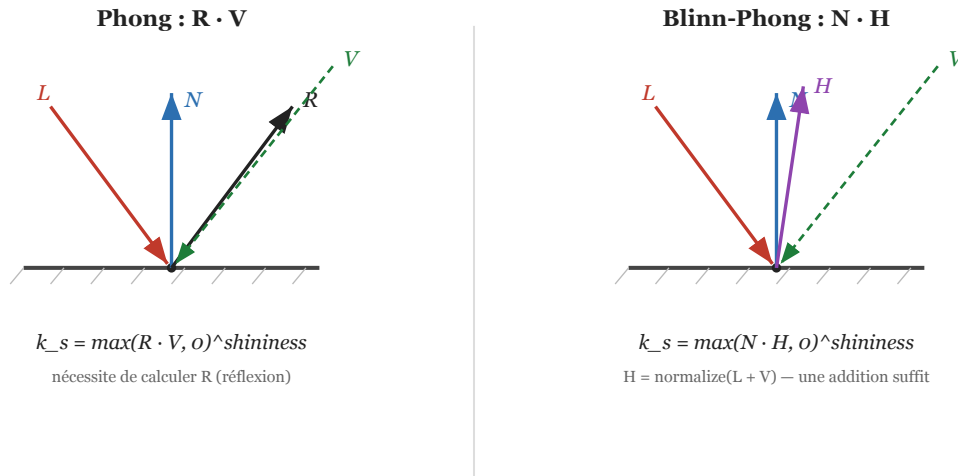
Le modèle Blinn-Phong remplace le vecteur réfléchi  $\vec{R}$  par le vecteur **half-angle**  $\vec{H}$  :

$$\vec{H} = \text{normalize}(\vec{L} + \vec{V})$$

$$k_s = \max(\vec{N} \cdot \vec{H}, 0)^{\text{shininess}}$$



**Avantage :**  $\vec{H}$  est plus simple à calculer que  $\vec{R}$  (pas besoin de réfléchir le vecteur — une addition + une normalisation suffit). C'est le modèle utilisé par défaut dans la plupart des moteurs temps réel, y compris le StandardMaterial de Babylon.js (voir Figure 7).



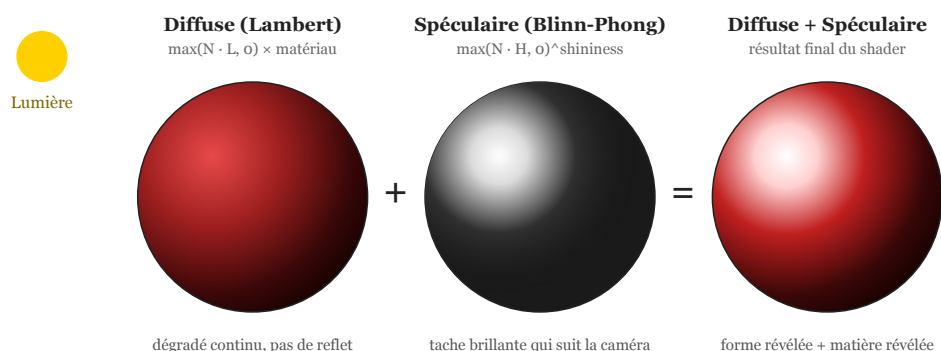
Blinn-Phong remplace R (coûteux) par  $H = \text{normalize}(L + V)$ . Visuellement équivalent, plus rapide.

Figure 7: Comparaison Phong vs Blinn-Phong. À gauche, on calcule le vecteur réfléchi  $\vec{R}$  et on le compare à  $\vec{V}$ . À droite, on calcule le vecteur half-angle  $\vec{H} = \text{normalize}(\vec{L} + \vec{V})$  et on le compare à  $\vec{N}$ .

### ❓ Pourquoi Blinn-Phong a remplacé Phong

Le produit scalaire  $\vec{N} \cdot \vec{H}$  donne un angle **deux fois plus petit** que  $\vec{R} \cdot \vec{V}$ . Pour retrouver le même reflet visuel, on multiplie généralement shininess par 4 quand on passe de Phong à Blinn-Phong. Le résultat est visuellement très proche, mais le calcul est plus rapide — un gain précieux quand on rend des millions de fragments par frame.

## 10. Le shader complet : diffus + spéculaire



La composante diffuse révèle la couleur et la forme ; la spéculaire révèle la brillance et la matière.

Figure 8: La composante diffuse (Lambert) révèle la couleur et la forme ; la composante spéculaire (Blinn-Phong) révèle la brillance et la matière. La somme donne le rendu final.

Fragment shader Phong (Lambert + spéculaire).

```

in vec3 vNormal;
in vec3 vLightDir;
in vec3 vViewDir;      // Direction de la caméra vers le fragment

uniform vec3 uLightColor;
uniform vec3 uMaterialColor;
uniform vec3 uSpecularColor; // Couleur du reflet (souvent blanc)
uniform float uShininess;    // Exposant de brillance

out vec4 fragColor;

void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);
    vec3 V = normalize(vViewDir);

    // Produit scalaire brut (non clampé) – sert pour diffuse ET pour le masque
    float NdotL = dot(N, L);

    // --- Composante diffuse (Lambert) ---
    float diffuse = max(NdotL, 0.0);

    // --- Composante spéculaire (Phong) ---
    vec3 R = reflect(-L, N);          //  $R = 2 \cdot (N \cdot L) \cdot N - L$ 
    // step(0.0, NdotL) vaut 1 si NdotL >= 0, sinon 0 → masque sans branchement
    float specular = pow(max(dot(R, V), 0.0), uShininess)
        * step(0.0, NdotL);

    // --- Combinaison ---
    vec3 color = uLightColor * uMaterialColor * diffuse
        + uLightColor * uSpecularColor * specular;

    fragColor = vec4(color, 1.0);
}

```

### ❗ Pourquoi `step()` plutôt qu'un `if` ?

Sans masque, la spéculaire apparaîtrait sur la face **non éclairée** — un reflet fantôme. Mais un `if` (`diffuse <= 0.0`) est un **branchement sur une valeur varying** : comme `diffuse` diffère d'un fragment à l'autre, les threads d'un même warp divergent et s'exécutent en série, ce qui annule l'avantage du SIMD.

La fonction `step(edge, x)` retourne 0 si  $x < \text{edge}$ , 1 sinon — c'est une opération **mathématique pure**, exécutée en lockstep par tous les threads. Multiplier la spéculaire par `step(0.0, NdotL)` coupe donc le reflet sur les faces sombres **sans aucune divergence**. C'est l'idiome standard en GLSL pour tout masque conditionnel.

*Variante Blinn-Phong (plus rapide).*

// ... même setup que ci-dessus ...

```

void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);
    vec3 V = normalize(vViewDir);

```

```

float NdotL = dot(N, L);

// --- Diffuse (Lambert) ---
float diffuse = max(NdotL, 0.0);

// --- Spéculaire (Blinn-Phong) ---
vec3 H = normalize(L + V); // Vecteur half-angle
float specular = pow(max(dot(N, H), 0.0), uShininess)
    * step(0.0, NdotL); // Masque sans branchement

vec3 color = uLightColor * uMaterialColor * diffuse
    + uLightColor * uSpecularColor * specular;

fragColor = vec4(color, 1.0);
}

```

## 11. StandardMaterial : assembler les deux en Babylon.js

### Deux approches d'éclairage

Babylon.js propose deux principaux types de matériaux pour interagir avec la lumière :

- **StandardMaterial** : Modèle empirique (Lambert + Blinn-Phong). Simple, rapide, mais peu réaliste. Propriétés : `diffuseColor`, `specularColor`, `specularPower`.
- **PBRMaterial** : Modèle physiquement correct (Physically Based Rendering). Plus coûteux, mais produit des résultats réalistes. Propriétés : `albedoColor`, `metallic`, `roughness`. (Vu en détail dans une session ultérieure.)

*StandardMaterial : Lambert + Blinn-Phong en une ligne.*

```

const mat = new BABYLON.StandardMaterial("mat", scene);

// Composante diffuse (Loi de Lambert)
mat.diffuseColor = new BABYLON.Color3(0.8, 0.2, 0.2); // Rouge

// Composante spéculaire (Blinn-Phong)
mat.specularColor = new BABYLON.Color3(1, 1, 1); // Reflet blanc
mat.specularPower = 32; // Shininess

// Quand vous réglez specularPower, vous contrôlez directement
// l'exposant uShininess du shader vu ci-dessus.

```

## Concepts clés à retenir

### 💡 Les 5 points essentiels

1. **La normale est reine** : Sans normales correctes, pas d'éclairage correct. Le produit scalaire  $\vec{N} \cdot \vec{L}$  est la formule fondamentale.
2. **Lambert = cosinus de l'angle** : L'intensité diffuse est  $\max(\vec{N} \cdot \vec{L}, 0)$ . Un simple produit scalaire de vecteurs normalisés donne le cosinus — c'est toute la magie.
3. **La couleur se multiplie** :  $\text{color}_{\text{final}} = \text{color}_{\text{light}} \times \text{color}_{\text{material}} \times \max(\vec{N} \cdot \vec{L}, 0)$ . Une lumière rouge sur un objet bleu donne du noir.
4. **Quatre lumières, quatre usages** : `HemisphericLight` (ambiance), `DirectionalLight` (soleil), `PointLight` (ampoule), `SpotLight` (projecteur).

5. **Spéculaire = reflet qui dépend de la caméra** : Contrairement au diffus, le spéculaire bouge quand on tourne autour de l'objet. Blinn-Phong  $(\vec{N} \cdot \vec{H})$  est l'optimisation standard de Phong  $(\vec{R} \cdot \vec{V})$ .

### 💡 Question de réflexion pour la prochaine session

“Nous savons maintenant éclairer des objets. Mais que se passe-t-il quand un objet bloque la lumière d'un autre ? Comment le GPU sait-il qu'un fragment est **derrière** un autre vu depuis la lumière ?”

**Réponse** : C'est le **Shadow Mapping**, que nous verrons en Session 6. Le GPU rend la scène depuis le point de vue de la lumière pour calculer ce qui est visible et ce qui est dans l'ombre.